

Bàn về thiết kế và ứng dụng trạng thái hợp lý trong tin học

Liu Yi

Trường Trung học số 3 Phúc Châu, Phúc Kiến

Trại đông Tin học toàn quốc, 2008

Nguồn gốc: Bàn về thiết kế và ứng dụng trạng thái hợp lý trong tin học

IOI China candidate team papers / 2008 / Day 1 / Liu Yi / state design and applications.doc

Tóm tắt

Phân tích và thiết kế trạng thái là một phần quan trọng trong tin học, xuất hiện trong quy hoạch động, tìm kiếm, liệt kê và nhiều thuật toán khác. Bài viết thảo luận vai trò của việc chọn trạng thái hợp lý qua ba ví dụ: *Square Root*, *Banal Tickets* và *Shoot Your Gun*. Các ví dụ cho thấy tối ưu thuật toán không chỉ là làm nhanh bước xử lý từng trạng thái; đôi khi việc giảm số lượng trạng thái mới là điểm quyết định.

Từ khóa: trạng thái; phân tích; tối ưu; cải tiến.

1 Dẫn nhập

Trong đời sống, thời gian hoàn thành công việc có thể viết dưới dạng:

$$\text{thời gian} = \text{số lượng công việc} \times \text{thời gian cho mỗi việc.}$$

Trong lập trình, một công thức tương tự là:

$$\text{thời gian chạy} = \text{số trạng thái} \times \text{thời gian xử lý mỗi trạng thái.}$$

Hai công thức trông giống nhau, nhưng điểm khác biệt là số lượng trạng thái không cố định. Cùng một bài toán, các cách biểu diễn trạng thái khác nhau có thể tạo ra số trạng thái rất khác nhau. Vì vậy ngoài việc tối ưu bước xử lý mỗi trạng thái, ta còn cần xét bản thân cách thiết kế trạng thái.

Một trạng thái tốt có thể:

- loại bỏ phần thông tin dư thừa;
- làm rõ bản chất bài toán;
- giảm độ khó cài đặt;
- biến một thuật toán tưởng như không khả thi thành khả thi.

Ba phần sau lần lượt trình bày ba hướng: phân tích số lượng trạng thái, cải tiến cách biểu diễn trạng thái, và thiết kế lại trạng thái khi cách thông thường thất bại.

2 Phân tích trạng thái hợp lý

Trước khi cài đặt, ta thường ước lượng độ phức tạp. Nếu ước lượng quá thấp, ta có thể lao vào một thuật toán không thể chạy kịp; nếu ước lượng quá cao, ta có thể bỏ qua một cách làm đơn giản nhưng đủ tốt. Ví dụ đầu tiên cho thấy việc đếm đúng số trạng thái có thể cứu một thuật toán tưởng như thô.

2.1 Ví dụ 1: Square Root

Với số nguyên a, n , cần tìm các x trong khoảng $0 < x < n$ sao cho

$$x^2 \equiv a \pmod{n}.$$

Mỗi truy vấn cho a, n , trong đó n là số nguyên tố, a nguyên tố cùng nhau với n , và $1 \leq a, n \leq 32767$. Số truy vấn có thể tới 100000.

Bài toán căn bậc hai modulo có các phương pháp số học chuyên dụng. Tuy nhiên, nếu không muốn dựa vào công thức sâu, ta thử cách liệt kê: với mỗi truy vấn, duyệt mọi x , kiểm tra $x^2 \bmod n$, rồi in các nghiệm. Một truy vấn tốn $\mathcal{O}(n)$, và tổng chi phí xấu nhất có thể quá lớn.

Nhưng cách nhìn đó lãng phí thông tin. Khi đã duyệt toàn bộ x cho một modulo n , ta không chỉ biết nghiệm của một giá trị a , mà biết nghiệm của mọi a theo cùng modulo. Vì vậy trước hết nhóm các truy vấn theo n . Có nhiều nhất 32767 giá trị n khác nhau.

Vẫn chưa đủ. Điều kiện đề bài cho n là số nguyên tố. Số số nguyên tố không vượt quá 32767 chỉ khoảng vài nghìn, nhỏ hơn nhiều so với toàn bộ miền. Do đó số trạng thái cần xử lý thực tế là số modulo nguyên tố xuất hiện, không phải số truy vấn nhân với n .

Thuật toán có thể làm như sau:

1. sàng các số nguyên tố tới 32767;
2. nhóm truy vấn theo modulo nguyên tố n ;
3. với mỗi n xuất hiện, duyệt $x = 1, \dots, n - 1$, đưa x vào danh sách nghiệm của $x^2 \bmod n$;
4. trả lời mọi truy vấn cùng modulo bằng bảng đã xây.

Nếu dùng sắp xếp hoặc bucket để nhóm truy vấn, chi phí chính là tổng các modulo nguyên tố thật sự xuất hiện. Bài toán vốn có vẻ cần lý thuyết số sâu lại được giải bằng liệt kê có tổ chức. Điểm chính là đếm đúng trạng thái: không phải “mỗi truy vấn là một thế giới riêng”, mà “mỗi modulo nguyên tố là một trạng thái lớn có thể tái sử dụng”.

3 Cải tiến biểu diễn trạng thái

Quy hoạch động thường được đánh giá theo hai phần: số trạng thái và chi phí chuyển trạng thái. Nhiều kỹ thuật tối ưu tập trung vào chuyển trạng thái, nhưng có những bài mà nút thắt nằm ở số trạng thái. Khi đó phải đổi cách biểu diễn.

3.1 Ví dụ 2: Banal Tickets

Một vé xe có $2N$ chữ số được gọi là thú vị nếu tích N chữ số đầu bằng tích N chữ số sau; ngược lại là tầm thường. Một số chữ số trên vé bị đục lỗ, ký hiệu bằng $?$. Cần đếm có bao nhiêu cách điền tạo vé thú vị và bao nhiêu cách tạo vé tầm thường, với $1 \leq N \leq 18$.

Hai số cần đếm là bù nhau, nên chỉ cần đếm vé thứ vị. Cách quy hoạch động đầu tiên là đặt $dp(i, p)$ là số cách điền i chữ số đầu để tích bằng p . Làm tương tự cho nửa sau rồi ghép theo cùng tích. Vấn đề là tích lớn nhất có thể rất lớn, không thể dùng trực tiếp làm chỉ số mảng.

Quan sát rằng mỗi chữ số chỉ thuộc $\{0, 1, \dots, 9\}$. Nếu tích khác 0, các thừa số nguyên tố có thể xuất hiện chỉ là

$$2, 3, 5, 7.$$

Vì vậy thay vì lưu tích p , ta lưu bộ bốn số mũ

$$(e_2, e_3, e_5, e_7),$$

tương ứng với

$$p = 2^{e_2} 3^{e_3} 5^{e_5} 7^{e_7}.$$

Trường hợp tích bằng 0 được tách riêng, vì không biểu diễn được bằng bộ số mũ hữu hạn.

Biểu diễn này giảm mạnh số trạng thái, nhưng vẫn còn dư thừa. Chẳng hạn, với độ dài cố định, không phải mọi bộ số mũ trong miền cực đại đều có thể được tạo từ tích các chữ số. Ví dụ, nếu số mũ của 2 đã đạt mức chỉ có thể do toàn bộ chữ số là 8, thì các số mũ của 3, 5, 7 không thể đồng thời dương.

Do đó bản gốc đề xuất sinh trước toàn bộ trạng thái thật sự có thể xuất hiện. Bắt đầu từ trạng thái đơn vị, dùng BFS/DFS nhân thêm một chữ số hợp lệ, thu được tập trạng thái khả dụng rồi ánh xạ bằng hash. Với N cực đại, số trạng thái từ khoảng hàng trăm nghìn giảm xuống dưới mức vài chục nghìn, đủ cho bộ nhớ 32MB nếu kết hợp với số nguyên lớn.

Ví dụ này cho thấy quá trình cải tiến trạng thái theo từng lớp:

- từ tích thô sang phân tích thừa số nguyên tố;
- từ không gian hình hộp của các số mũ sang chỉ lưu trạng thái thật sự đạt được;
- tách riêng trạng thái tích 0.

4 Thiết kế lại trạng thái

Không phải lúc nào một mô hình kinh điển cũng dùng được trực tiếp. Khi số trạng thái quá lớn, đôi khi cần quay lại bản chất hình học hoặc vật lý của bài toán để thiết kế trạng thái khác.

4.1 Ví dụ 3: Shoot Your Gun

Bài *Shoot Your Gun* có ba đa giác trục giao: G , T và đa giác lớn M . Ta đặt súng tại một điểm bất kỳ trên biên G , bắn theo một trong bốn hướng chéo. Tia đạn phản xạ trên biên M theo quy tắc gương. Cần số lần phản xạ ít nhất để chạm biên T , trước khi chạm lại G .

Khó khăn đầu tiên là điểm bắn trên biên G có vô hạn lựa chọn. Tuy nhiên, với các điểm không phải đỉnh trên cùng một cạnh, nếu tia bắn không gặp đỉnh đặc biệt, quỹ đạo phản xạ có cùng số lần phản xạ. Vì thế có thể chọn một điểm đại diện, chẳng hạn trung điểm cạnh. Cùng với các đỉnh nguyên, số trạng thái trở thành hữu hạn.

Nếu mô hình hóa mỗi ô lưới, mỗi điểm và bốn hướng bắn thành trạng thái, số trạng thái vẫn rất lớn vì tọa độ có thể tới vài nghìn. Hơn nữa, nhiều điểm được biểu diễn lặp: góc phải dưới của một ô cũng là góc trái trên của ô kề. Loại bỏ biểu diễn trùng giảm được một phần nhưng chưa đủ.

Điều kiện quan trọng là tia sáng đi thẳng cho tới khi gặp biên đa giác; chỉ ở biên mới có lựa chọn phản xạ. Do đó các điểm nằm giữa đường đi không cần trở thành trạng thái. Ta chỉ cần xét các điểm trên

biên của các đa giác và hướng tới tiếp theo. Số trạng thái khi đó cùng bậc với số điểm biên, nhỏ hơn rất nhiều so với toàn bộ lưới.

Một cách làm là xây đồ thị trạng thái rồi chạy SPFA hoặc Dijkstra, trong đó mỗi cạnh tương ứng với một đoạn tia tới lần phản xạ kế tiếp. Nhưng bản gốc còn đầy xa hơn: các đường sáng không giao nhau một phần; hoặc không trùng, hoặc trùng hoàn toàn. Vì vậy có thể mô phỏng trực tiếp từng tia xuất phát.

Quy trình:

1. liệt kê điểm xuất phát đại diện trên biên G và bốn hướng;
2. từ điểm hiện tại, tìm cạnh biên gần nhất mà tia sẽ gặp;
3. phản xạ tia trên cạnh ấy, cập nhật điểm và hướng;
4. nếu chạm T , cập nhật đáp án; nếu chạm lại G , dừng tia này;
5. lặp cho tới khi tia kết thúc hoặc trạng thái đã xét lặp lại.

Số điểm biên trong trường hợp xấu vẫn ở mức có thể chấp nhận được, và mỗi hướng xuất phát chỉ cần xét một số lần hữu hạn. Cài đặt vì vậy đơn giản hơn nhiều so với xây đồ thị đầy đủ.

Điểm rút ra là: thiết kế trạng thái không nên máy móc theo ô lưới. Trạng thái phải đặt tại nơi quỹ đạo thật sự thay đổi, tức ở biên phản xạ.

5 Tổng kết

Ba ví dụ trên đều xoay quanh một câu hỏi: ta đang xử lý bao nhiêu trạng thái, và các trạng thái ấy có thật sự cần thiết không?

Trong *Square Root*, phân tích lại miền trạng thái cho thấy thuật toán liệt kê có thể dừng được nếu nhóm theo modulo nguyên tố. Trong *Banal Tickets*, thay đổi biểu diễn tích thành bộ số mũ nguyên tố rồi hash các trạng thái thật sự xuất hiện giúp quy hoạch động vượt qua giới hạn bộ nhớ. Trong *Shoot Your Gun*, bỏ mô hình ô lưới và chỉ đặt trạng thái ở biên phản xạ làm thuật toán vừa đơn giản vừa nhanh hơn.

Thiết kế trạng thái hợp lý giúp ta hóa phức thành giản, giảm số lượng tính toán, và làm rõ mạch suy luận. Muốn làm được điều đó cần phân tích kỹ điều kiện đề bài, tổ chức lại thông tin, và tích lũy kinh nghiệm từ nhiều dạng bài.

Lời cảm ơn

Tác giả cảm ơn cô Wei Lizhen của Trường Trung học số 3 Phúc Châu đã hướng dẫn; huấn luyện viên Liu Rujia đã góp ý chỉnh sửa; Yang Mu từ Đại học Thanh Hoa, Wang Hang từ Đại học Giao thông Thượng Hải, và Sun Linchun đã hỗ trợ trong quá trình hoàn thiện bài viết.

References

[1] Liu Rujia và Huang Liang, *Nghệ thuật thuật toán và thi tin học*.

Ghi chú về phụ lục

Bản gốc kèm phát biểu đầy đủ của ba bài *Square Root*, *Banal Tickets*, *Shoot Your Gun* và các chương trình Pascal mẫu. Bản dịch đã rà lại cả ba ví dụ trong thân bài: miền modulo nguyên tố của *Square*

Root, biểu diễn tích bằng bộ số mũ trong *Banal Tickets*, và trạng thái biên phản xạ trong *Shoot Your Gun*. Không còn tham chiếu hình hoặc chú thích treo. Các chương trình Pascal mẫu được giữ theo ngoại lệ mã mẫu: không chép nguyên văn, chỉ đưa những chi tiết cần cho phân tích thuật toán vào phần tương ứng.